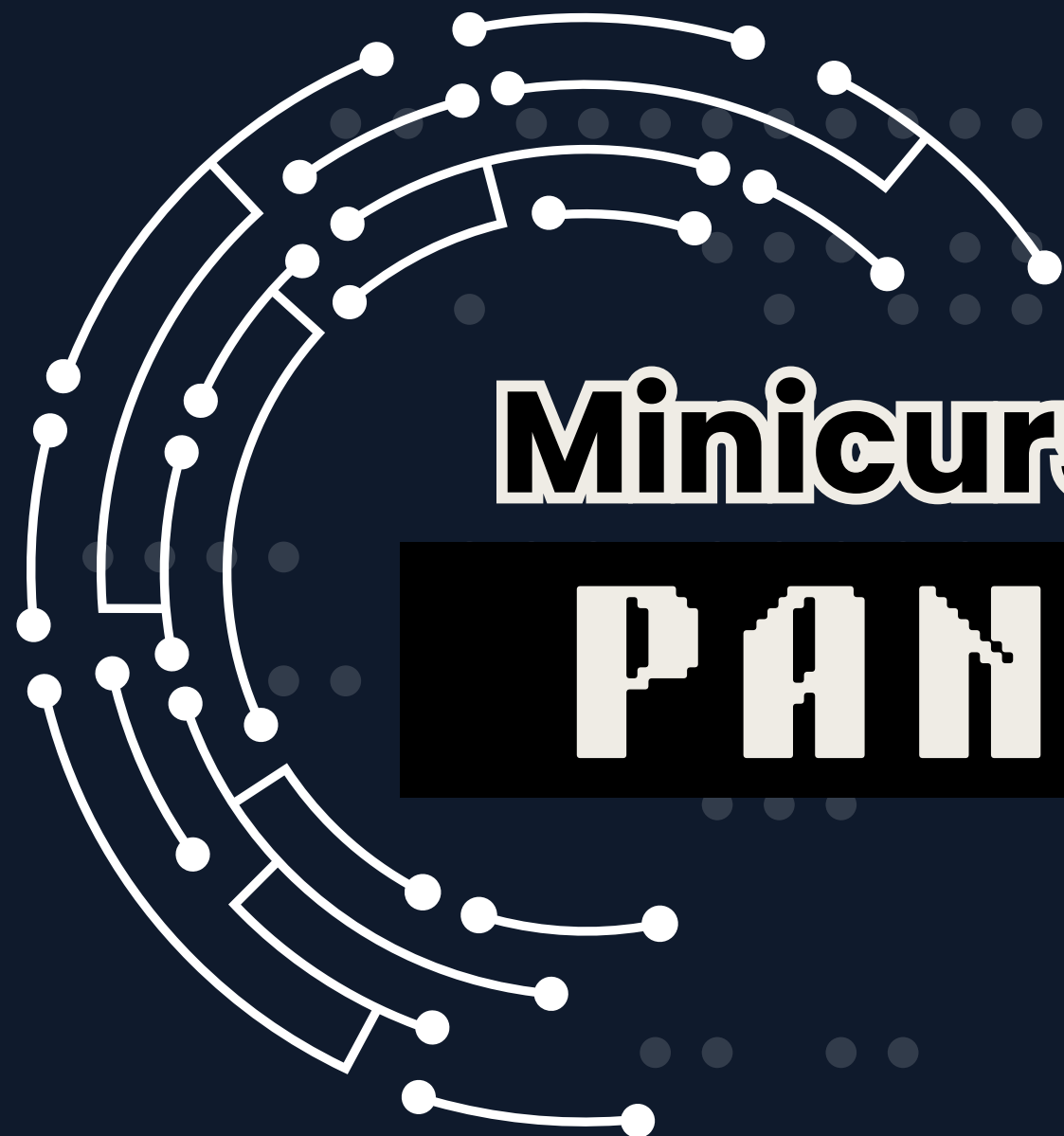




Minicurso de

PANDAS



INTRODUÇÃO

- **NUMPY**

Utilizada para vetores e operações numéricas

- **PANDAS**

Utilizada para dados tabulares

- **MATPLOTLIB**

Utilizada para visualização de dados

OBJETIVOS

- Entender a relação entre Python, NumPy e pandas;
- Carregar e inspecionar dados;
- Selecionar, filtrar e alterar um DataFrame;
- Trabalhar com dados faltantes;
- Agregar com groupby;
- Combinar tabelas com merge e concat;
- Integrar pandas com NumPy e Matplotlib;
- Salvar resultados em CSV e conhecer outros formatos.

PRIMEIROS PASSOS



```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt
```

LISTA VS ARRAY

- **LISTA**

Cada elemento pode ser de um tipo diferente, guardado separadamente na memória.

- **ARRAY**

Todos os elementos são do mesmo tipo, guardados lado a lado num bloco só de memória

LISTA VS ARRAY



```
lista = [10, 20, 30, 40]  
arr = np.array(lista)
```

```
print("Lista:", lista)  
print("Lista * 2:", lista * 2)
```

```
print("Array:", arr)  
print("Array * 2:", arr * 2)
```

LISTA VS ARRAY

```
... Lista: [10, 20, 30, 40]  
Lista * 2: [10, 20, 30, 40, 10, 20, 30, 40]  
Array: [10 20 30 40]  
Array * 2: [20 40 60 80]
```

DICIONÁRIO PARA DATAFRAME



```
dados = {  
    "nome": ["Ana", "Bruno", "Carlos"],  
    "idade": [25, 32, 28]  
}
```

```
df_pequeno = pd.DataFrame(dados)  
df_pequeno
```

DICIONÁRIO PARA DATAFRAME

...

	<i>nome</i>	<i>idade</i>
0	Ana	25
1	Bruno	32
2	Carlos	28

SÉRIE VS DATAFRAME

- **SÉRIE**

Cada valor é associado a um identificador, que serve para acessá-lo

- **DATAFRAME**

Várias colunas alinhadas pelo mesmo índice, formando uma tabela

SÉRIE VS DATAFRAME



```
serie_idade = df_pequeno["idade"]  
  
print(type(serie_idade))  
serie_idade
```

SÉRIE VS DATAFRAME

...

idade

0	25
1	32
2	28

ABRINDO O CSV



```
df = pd.read_csv("dataset.csv")  
df.head()
```

INSPEÇÃO INICIAL

- **DF.HEAD()** E **DF.TAIL()**

Mostra as primeiras e últimas linhas do DataFrame

- **DF.SHAPE**

Mostra o tamanho (linhas, colunas)

- **DF.COLUMNS**

Lista o nome das colunas

INSPEÇÃO INICIAL

- **DF.INFO()**

Faz um resumo geral (tipos de dados, variáveis nulas, memória)

- **DF.DESCRIBE()**

Mostra as estatísticas descritivas das colunas numéricas.

TIPOS DE DADOS

- `PD.TO_DATETIME(DF["DATA"])`

Converte texto em data de verdade, permitindo operações de data

- `.DT`

Acessor que extrai partes da data (year, month, day_name())/

- `DF[...].HEAD()`

Seleciona um conjunto de linhas e mostra as primeiras linhas.

TIPOS DE DADOS



```
df["data"] = pd.to_datetime(df["data"])  
df.dtypes
```



```
df["ano"] = df["data"].dt.year  
df["mes"] = df["data"].dt.month  
df["dia_semana"] = df["data"].dt.day_name()  
  
df[["data", "ano", "mes", "dia_semana"]].head()
```

OPERAÇÕES

- **SUM**

Retorna a soma de todos os valores pertencentes à coluna desejada

- **MEAN**

Retorna a média da coluna desejada

- **MEDIAN**

Retorna a mediana da coluna desejada

OPERAÇÕES

- **MIN**

Retorna o menor valor presente na coluna desejada

- **MAX**

Retorna o maior valor presente na coluna desejada

OPERAÇÕES

- **DF["PRODUTOS"]**

Acessa uma única coluna, retorna uma Series

- **DF[["PRODUTO"]]**

Acessa uma coluna dentro de uma lista, retorna uma DataFrame

- **DF[["PRODUTO", "QUANTIDADE"]]**

Acessa várias colunas de uma vez

OPERAÇÕES

- `DF[:5]`

Pega as 5 primeiras linhas

- `DF.ILOC[0:5, 0:4]`

Seleciona por posição, linhas de 0-4 e colunas de 0-3

- `DF.LOC[0:4, ["DATA", "PRODUTO"]]`

Seleciona por identificador, linhas de indice 0 a 4 e colunas pelo nome

FILTROS BOOLEANOS

- **DF[COND]**

Filtra linhas que atendem uma condição

- **& | -**

Combinam condições(E, OU, NÃO), cada uma entra parênteses

FILTROS BOOLEANOS

- `.ISIN([...])`

Filtra valores dentro de uma lista

- `.BETWEEN(A,B)`

Filtra valores dentro de um intervalo

CRIANDO E ALTERANDO COLUNAS

- `DF["FATURAMENTO"] = DF["QUANTIDADE"] * DF["PRECO"]`

Cria uma coluna nova multiplicando duas colunas existentes, elemento a elemento vetorizado.

- `DF["TICKET_ALTO"] = DF["FATURAMENTO"]
 >= 3000`

Cria coluna booleana (True/False) a partir de uma condição.

`.copy()` E SUBCONJUNTOS

- `.copy()`

Cria um subconjunto independente do DataFrame original, evitando alterar sem querer o df original ao modificar o subconjunto.

DAADOS FALTANTES

- **ISNA()**

Verifica se o data frame possuiu algum valor ausente

- **FILLNA()**

Substituiu valores ausentes por um valor fixo

- **DROPNA()**

Remove linhas ou colunas que possuem valores ausentes

ORDENAÇÃO E VALORES ÚNICOS

- **DF.SORT_VALUES()**

Ordena os valores do dataframe ou da série, podendo ser de diversas maneiras diferentes

- **DF[...].VALUE_COUNTS()**

Conta quantas vezes cada valor únicos aparece em uma coluna ou série

- **DF[...].UNIQUE()**

Retorna um array com os valores distintos de uma coluna ou série, mantendo a ordem em que eles aparecem

GROUPBY

- **GROUPBY**

Agrupa linhas por valores de uma ou mais colunas.

- **DF.GROUPBY("CIDADE")["PREÇO"]**

Agrupa por cidades e seleciona a coluna preço

- **DF.GROUPBY("PRODUTO")["QNTD"].SUM()**

Soma quantidade por produto

GROUPBY

- `DF.GROUPBY("CIDADE", DROPNA=FALSE)`

Agrupa por cidade mantendo os valores nulos com grupo

- `DF.AGG(NOME={"COLUNA", "FUNCAO"})`

Aplica várias agregações de uma vez, nomeando cada coluna resultante.

- `DF.GROUPBY(["CIDADE", "PRODUTO"])[...]`

Agrupa por mais de uma coluna ao mesmo tempo.

NUMPY COM PANDAS

- `NP.LOG(DF["PRECO"])`

Aplica logaritmo em cada valor da coluna, criando coluna nova.

- `NP.WHERE(COND, "ALTO", "BAIXO")`

Cria coluna com valores condicionais (tipo um "se/senão" vetorizado).

APPLY

- **APPLY**

Aplica uma função Python própria a cada elemento de uma Series, útil quando não existe operação vetorizada pronta pra aquilo.

- **DF["QNTD"].APPLY(CLASSIFICAR_QNTD)**

Passa cada valor da coluna pela função, retorna coluna nova com o resultado.

RENOMEAR E REMOVE

- `DF.RENAME(COLUMNS={"ANTIGO": "NOVO"})`

Renomeia colunas.

- `DF.DROP(COLUMNS=[...])`

Remove colunas.

- `DF.DROP_DUPLICATES()`

Remove as linhas duplicadas.

STRINGS

- **DF.STR**

Acessor especial de colunas de texto, dá acesso a operações vetorizadas sobre string.

- **DF["CLIENTE"].STR.UPPER()**

Transforma o texto da coluna em maiúsculo, cria coluna nova.

- **.STR.CONTAINS("01",CASE=FALSE, NA=FALSE)**

Filtra linhas cujo texto contém aquele trecho, ignorando maiúscula/minúscula e valores nulos.

EMPILHAR TABELAS

- **CONCAT**

Empilha tabelas uma embaixo da outra (junta linhas).

- **`PD.CONCAT([DF1, DF2], IGNORE_INDEX=True)`**

Junta os DataFrames em um só, e `ignore_index=True` recria o índice do zero, sem repetir os antigos.

COMBINAR TABELAS

- **MERGE**

Combina duas tabelas com base numa coluna em comum

- **PD.MERGE(DF, VENDEDORES,
ON="VENDEDOR", HOW="LEFT")**

Junta df com vendedores pela coluna "vendedor".

On: Define a coluna usada para fazer o cruzamento

How="left": Mantém todas as linhas de df, buscando correspondência em vendedores

PANDAS + MATPLOTLIB

- `PLT.FIGURE(FIGSIZE=(8, 4))`

Define o tamanho da figura.

- `PLT.HIST(DF["PREÇO"], BINS=15)`

Plota o histograma dos preços

PANDAS + MATPLOTLIB

- **PLT.XLABEL, PLT.YLABEL, PLT.TITLE**

Definem os rótulos dos eixos e o título.

- **PLT.SHOW()**

Exibe o gráfico

- **DF.GROUPBY("PRODUTO")
["FATURAMENTO"].SUM().SORT_VALUES(AS
CENDING=FALSE)**

Soma o faturamento por produto

PANDAS + MATPLOTLIB

- `PLOT(KIND="BAR")`

Gera gráfico de barras a partir do resultado.

- `PLT.XTICKS(ROTATION=45)`

Gira os rótulos do eixo x em 45°.

- `PLT.TIGHT_LAYOUT()`

Ajusta o espaçamento pra nada ficar cortado.

COMO SALVAR E EXPORTAR

- `PLT.SAVEFIG("ARQUIVO.JPG")`

Exporta a imagem para o diretório raiz

- `DF.TO_TIPO("ARQUIVO.TIPO")`

Exporta a tabela como o tipo escolhido (.csv, .xlsx, etc...)

OBRIGADO PELA AULA!
DÚVIDAS?

NOS AVALIE! É MUITO IMPORTANTE

